



Đề tài đồ án NT547 - Các đồ án

Blockchain (Trường Đại Học Công Nghệ Thông Tin)



messages.pdf_cover_qr_code_label

Đồ Án Môn Học Blockchain: Nền tảng, ứng dụng và bảo mật – NT547.P11.ANTT

Đồ án môn học sẽ được chia thành hai nhánh chính, mỗi nhánh đều có độ khó và yêu cầu cao như nhau:

- Nhánh Nghiên cứu (Research Track):** Nhánh này dành cho những sinh viên có đam mê tìm hiểu sâu về các ý tưởng học thuật tiên tiến, muốn rèn luyện kỹ năng đọc hiểu, phân tích và tái hiện các công trình khoa học. Mục tiêu không chỉ là lập trình, mà là thẩm thấu, kiểm chứng và phê bình một ý tưởng mới từ một bài báo khoa học đã được công bố. Con đường này giúp mài giũa tư duy phản biện, khả năng nghiên cứu độc lập và thực nghiệm khoa học – những kỹ năng tối quan trọng cho các vị trí R&D trong công nghiệp hoặc con đường học thuật sau này.
- Nhánh Phát triển (Development Track):** Nhánh này là một thử thách về kỹ thuật, dành cho những sinh viên muốn xây dựng các công cụ và ứng dụng thực tế, có giá trị sử dụng. Mục tiêu là tạo ra một sản phẩm phần mềm hoàn chỉnh, được thiết kế tốt, giải quyết một vấn đề cụ thể trong không gian Web3. Con đường này giúp rèn luyện các kỹ năng về kiến trúc phần mềm, tích hợp API, trải nghiệm người dùng và triển khai sản phẩm – những kỹ năng cốt lõi của một kỹ sư blockchain chuyên nghiệp.

Cả hai nhánh đều đòi hỏi sự nỗ lực, sáng tạo và một sự hiểu biết sâu sắc về các nội dung đã học. Đồ án này không chỉ là một bài tập cuối kỳ, mà sẽ là một sản phẩm trong portfolio của các em, một minh chứng rõ ràng cho năng lực và đam mê của các em đối với lĩnh vực đầy tiềm năng này.

Nhánh I: Research Track

Tổng quan các Đề tài Nhánh Nghiên cứu

Mã Đề Tài	Tên Đề Tài	Lĩnh Vực Cốt Lõi
R1	Phát hiện Lỗ hổng Hợp đồng Thông minh sử dụng Mạng Nơ-ron Đồ thị (GNN)	Phân tích Bảo mật, Học máy
R2	Xây dựng Giao thức Bỏ phiếu Ẩn danh Bảo toàn Quyền riêng tư với Bảng chứng không Tri thức (ZKP)	Mật mã học Ứng dụng, ZKP
R3	Phân tích và Triển khai Cơ chế Thu hồi Chứng chỉ Xác thực (VC) Bảo	Danh tính Phi tập trung (DID), Mật mã học

	toàn Quyền riêng tư	
R4	Triển khai Giao thức Giao tiếp Liên-Shard (Cross-Shard) sử dụng Bảng chứng không Tri thức	Khả năng Mở rộng (Scalability), ZKP
R5	Kết hợp Mạng Nơ-ron Đồ thị và Phân tích Tĩnh để Tăng cường Phát hiện Lỗ hổng	Phân tích Bảo mật, Học sâu
R6	Phân tích So sánh các Cơ chế Đấu giá MEV (Maximal Extractable Value)	Kinh tế học Tiền mã hóa (Cryptoeconomics), DeFi
R7	Triển khai zk-ML: Thực thi Mô hình Học máy có thể Kiểm chứng trên Blockchain sử dụng ZKP	ZKP, Học máy, Quyền riêng tư
R8	Mô phỏng và Đánh giá Tấn công Double-Spending trong Giao thức Sharding dựa trên Lưu trữ Đa-Shard	Bảo mật Blockchain, Sharding
R9	Phân tích và Tái hiện Mô hình AMM Thanh khoản Tập trung (Concentrated Liquidity)	DeFi, AMM
R10	Mã hóa Dựa trên Danh tính Phân cấp Ẩn danh cho việc Thu hồi VC	Danh tính Phi tập trung (DID), Mật mã học Nâng cao

Chi tiết các Đề tài Nghiên cứu

R1: Phát hiện Lỗ hổng Hợp đồng Thông minh sử dụng Mạng Nơ-ron Đồ thị (GNN)

- **Bài Báo Khoa Học Chính:** ijcai.org/proceedings/2020/0454.pdf
- **Bối Cảnh Vấn Đề:** Các công cụ phân tích bảo mật hợp đồng thông minh truyền thống thường dựa vào các quy tắc được định nghĩa sẵn bởi chuyên gia (rule-based). Phương pháp này có nhiều hạn chế: chúng dễ bị bỏ sót các lỗ hổng phức tạp, tạo ra nhiều kết quả dương tính giả (false positive) và âm tính giả (false negative), và kẻ tấn công có thể dễ dàng "lách luật" để qua mặt các quy tắc này. Trong bối cảnh hàng tỷ đô la đang được quản lý bởi các hợp đồng thông minh, việc có một phương pháp phát hiện lỗ hổng mạnh mẽ và tự động hơn là cực kỳ cấp thiết. Sự chuyển dịch từ các hệ thống dựa trên luật sang

các hệ thống dựa trên học máy đang là một xu hướng lớn trong toàn bộ ngành kỹ thuật phần mềm, và blockchain cũng không ngoại lệ.

- **Đóng Góp Chính của Bài Báo:** Bài báo này đề xuất một cách tiếp cận mới: biểu diễn mã nguồn của một hàm trong hợp đồng thông minh dưới dạng một "đồ thị hợp đồng" (contract graph). Đồ thị này không chỉ nắm bắt cấu trúc cú pháp mà còn cả các mối quan hệ ngữ nghĩa quan trọng như luồng điều khiển (control flow) và luồng dữ liệu (data flow) giữa các câu lệnh. Sau đó, một mô hình Mạng Nơ-ron Đồ thị (cụ thể là DR-GCN) được áp dụng trên đồ thị này để tự động học các mẫu cấu trúc đặc trưng cho các lỗ hổng phổ biến như Tấn công Tái nhập (Reentrancy) và Phụ thuộc Dấu thời gian (Timestamp Dependence).
- **Nhiệm Vụ của Sinh Viên:**
 1. **Nghiên cứu Lý thuyết:** Đọc và hiểu sâu bài báo và các kho mã nguồn liên quan. Nắm vững các khái niệm về GNN, luồng điều khiển, luồng dữ liệu trong chương trình.
 2. **Xây dựng Đồ thị:** Đây là phần thách thức nhất. Sinh viên cần viết một kịch bản (script) để phân tích mã nguồn Solidity (sử dụng một thư viện parser như @solidity-parser/parser) và tự động trích xuất, xây dựng các "đồ thị hợp đồng" theo đúng mô tả trong bài báo. Các nút trong đồ thị sẽ đại diện cho các biến hoặc lệnh gọi hàm quan trọng, và các cạnh sẽ biểu diễn mối quan hệ phụ thuộc giữa chúng.
 3. **Triển khai Mô hình:** Cài đặt lại mô hình DR-GCN bằng PyTorch hoặc TensorFlow, tuân thủ kiến trúc được mô tả trong bài báo.
 4. **Huấn luyện và Đánh giá:** Sử dụng bộ dữ liệu được cung cấp trong các kho mã nguồn tham khảo để huấn luyện mô hình. Sau đó, tiến hành đánh giá và cố gắng tái hiện lại các chỉ số hiệu suất (accuracy, precision, recall, F1-score) được báo cáo trong bài báo.
 5. **Viết Báo cáo:** Soạn một báo cáo chi tiết, trình bày lại lý thuyết, mô tả quá trình triển khai, phân tích các kết quả thực nghiệm và đưa ra những nhận xét, đánh giá của nhóm về phương pháp này.
- **Kiến thức Nền tảng Yêu cầu:** Kiến thức cơ bản về Học máy, Mạng Nơ-ron. Kinh nghiệm làm việc với PyTorch/TensorFlow là một lợi thế. Kỹ năng lập trình Python tốt.

R2: Xây dựng Giao thức Bỏ phiếu Ẩn danh Bảo toàn Quyền riêng tư với Bảng chứng không Tri thức (ZKP)

- **Bài Báo Khoa Học Chính:** Đề tài này dựa trên các nguyên lý và kiến trúc của các hệ thống tiên tiến như (<https://semaphore.appliedzkp.org/>) và được mô tả trong các bài khảo sát.
- **Bối Cảnh Vấn Đề:** Các hệ thống bỏ phiếu điện tử truyền thống đối mặt với một nghịch lý nan giải: làm thế nào để vừa đảm bảo tính minh bạch, có thể kiểm chứng của cuộc bầu cử (mọi lá phiếu hợp lệ đều được đếm đúng), vừa bảo vệ quyền riêng tư và sự ẩn danh của cử tri (không ai biết ai đã bỏ phiếu cho ai). Blockchain cung cấp một hòm phiếu công khai, minh bạch và bất biến, nhưng việc đảm bảo ẩn danh đòi hỏi các kỹ thuật mật mã tiên tiến.

- **Đóng Góp Chính của Bài Báo (Lý thuyết):** Bằng chứng không Tri thức (Zero-Knowledge Proofs), đặc biệt là zk-SNARKs, cung cấp một giải pháp mạnh mẽ cho vấn đề này. ZKP cho phép một người (prover) chứng minh cho người khác (verifier) rằng họ biết một thông tin bí mật nào đó mà không cần tiết lộ chính thông tin đó. Trong bối cảnh bỏ phiếu, một cử tri có thể tạo ra một bằng chứng để chứng minh hai điều: (1) họ thuộc nhóm những người đủ tư cách bỏ phiếu, và (2) lá phiếu đã mã hóa của họ là hợp lệ (ví dụ: chỉ chọn 1 trong số N ứng viên), mà không tiết lộ danh tính hay lựa chọn của mình. ZKP là một công cụ mật mã tổng quát giúp tách rời quyền hạn (permission) khỏi danh tính (identity), một khái niệm có ứng dụng sâu rộng vượt ra ngoài việc bỏ phiếu, như trong các đợt airdrop riêng tư, mạng xã hội ẩn danh, hay chia sẻ dữ liệu bí mật.
- **Nhiệm Vụ của Sinh Viên:**
 1. **Tìm hiểu ZKP:** Nghiên cứu các khái niệm cơ bản về zk-SNARKs, cách chúng hoạt động, và các thuộc tính như tính toàn vẹn (completeness), tính đúng đắn (soundness) và không tri thức (zero-knowledge).
 2. **Thiết kế Mạch ZKP (Circuit):** Sử dụng một framework như Circom để thiết kế một mạch ZKP đơn giản. Mạch này sẽ nhận đầu vào là một khóa bí mật của cử tri và lựa chọn của họ. Nó sẽ chứng minh rằng khóa công khai tương ứng với khóa bí mật này nằm trong một cây Merkle chứa danh sách tất cả các cử tri hợp lệ, và lựa chọn bỏ phiếu là một giá trị hợp lệ (ví dụ, 0 hoặc 1).
 3. **Phát triển Hợp đồng Thông minh:** Viết một hợp đồng thông minh trên Solidity để quản lý cuộc bỏ phiếu. Hợp đồng này sẽ lưu trữ Merkle root của danh sách cử tri, định nghĩa các ứng viên, và có một hàm để nhận lá phiếu. Hàm này sẽ nhận vào một ZKP, một mã định danh rỗng (nullifier) để chống bỏ phiếu hai lần, và lá phiếu đã được mã hóa. Hợp đồng sẽ xác minh ZKP, nếu hợp lệ, nó sẽ ghi nhận lá phiếu.
 4. **Xây dựng Prover Phía Client:** Viết một kịch bản (ví dụ, bằng JavaScript với thư viện snarkjs) để mô phỏng một cử tri. Kịch bản này sẽ lấy thông tin của cử tri, tạo ra bằng chứng ZKP và gửi giao dịch đến hợp đồng thông minh.
 5. **Viết Báo cáo:** Tài liệu hóa toàn bộ thiết kế giao thức, chi tiết về mạch ZKP, logic của hợp đồng thông minh, và phân tích các đảm bảo an ninh mà hệ thống cung cấp (ẩn danh, không thể bỏ phiếu hai lần, có thể kiểm chứng công khai).
- **Kiến thức Nền tảng Yêu cầu:** Hiểu biết về các cấu trúc dữ liệu (cây Merkle). Có khả năng tư duy logic và thuật toán tốt. Sẵn sàng tìm hiểu các khái niệm mật mã học mới. Kinh nghiệm với Solidity và JavaScript.

R3: Phân tích và Triển khai Cơ chế Thu hồi Chứng chỉ Xác thực (VC) Bảo toàn Quyền riêng tư

- **Bài Báo Khoa Học Chính:** [\[2503.22010\] Towards Privacy-Preserving Revocation of Verifiable Credentials with Time-Flexibility](#)
- **Bối Cảnh Vấn Đề:** Trong hệ thống Danh tính Tự chủ (Self-Sovereign Identity - SSI), người dùng kiểm soát danh tính và dữ liệu của mình thông qua các Chứng chỉ Xác thực (Verifiable Credentials - VCs). Tuy nhiên, một VC có thể cần bị thu hồi trước khi hết hạn

(ví dụ: bằng lái xe bị treo, bằng cấp bị thu hồi). Việc kiểm tra trạng thái thu hồi này đặt ra một vấn đề lớn về quyền riêng tư: nếu mỗi lần một VC được trình diện, bên xác minh (Verifier) lại truy vấn đến bên phát hành (Issuer), thì Issuer có thể theo dõi mọi hoạt động của người dùng, phá vỡ nguyên tắc cốt lõi của SSI.

- **Đóng Góp Chính của Bài Báo:** Bài báo này đề xuất một phương pháp mới để thu hồi VC mà vẫn bảo vệ quyền riêng tư của người dùng (Holder). Ý tưởng chính là sử dụng một kỹ thuật mật mã gọi là Mã hóa Dựa trên Danh tính Phân cấp Ẩn danh (Anonymous Hierarchical Identity-Based Encryption). Cơ chế này cho phép Holder cấp quyền cho Verifier để kiểm tra thông tin thu hồi chỉ trong một khoảng thời gian nhất định (ví dụ, chỉ tại thời điểm trình diện VC). Điều này ngăn cản Verifier theo dõi trạng thái của VC theo thời gian và cũng ngăn Issuer biết được khi nào và ở đâu VC được sử dụng.
- **Nhiệm Vụ của Sinh Viên:**
 1. **Nghiên cứu Lý thuyết:** Đọc kỹ bài báo để hiểu sâu về mô hình mối đe dọa (threat model) và các mục tiêu về quyền riêng tư (ví dụ: untraceability by Verifier/Issuer). Tìm hiểu các khái niệm cơ bản về Mã hóa Dựa trên Danh tính (IBE).
 2. **Thiết kế Giao thức:** Dựa trên mô tả trong bài báo, đặc tả lại các bước của giao thức một cách chi tiết: quá trình thiết lập ban đầu của Issuer, quá trình Holder nhận VC và khóa, quá trình Holder ủy quyền cho Verifier, và quá trình Verifier kiểm tra trạng thái thu hồi.
 3. **Triển khai Proof-of-Concept:** Cài đặt một phiên bản mô phỏng của giao thức bằng một ngôn ngữ lập trình phù hợp (ví dụ: Python với các thư viện mật mã). Sinh viên không cần triển khai lại toàn bộ thuật toán mã hóa phức tạp từ đầu, mà có thể sử dụng các thư viện có sẵn hoặc mô phỏng các hộp đen (black-box) mật mã, tập trung vào logic của giao thức.
 4. **Phân tích An ninh và Quyền riêng tư:** Phân tích xem giao thức đã triển khai có đáp ứng được các mục tiêu đã đề ra trong bài báo hay không. Ví dụ: chứng minh rằng Verifier không thể kiểm tra trạng thái thu hồi ngoài khung thời gian được cho phép, và Issuer không thể liên kết một yêu cầu kiểm tra với một lần trình diện VC cụ thể.
 5. **Viết Báo cáo:** Trình bày lại các phương pháp thu hồi VC hiện có và các vấn đề của chúng. Mô tả chi tiết giải pháp được đề xuất trong bài báo, quá trình triển khai mô phỏng của nhóm, và kết quả phân tích an ninh, quyền riêng tư.
- **Kiến thức Nền tảng Yêu cầu:** Hiểu biết tốt về các nguyên tắc bảo mật và quyền riêng tư. Có kiến thức cơ bản về mật mã khóa công khai. Kỹ năng lập trình tốt.

R4: Triển khai Giao thức Giao tiếp Liên-Shard (Cross-Shard) sử dụng Bảng chứng không Tri thức

- **Bài Báo Khoa Học Chính:** Dựa trên các ý tưởng trong [Monoxide: Scale out Blockchains with Asynchronous Consensus Zones | USENIX](#) và các kỹ thuật được mô tả trong các bài khảo sát về ZKP và sharding. [Link](#).
- **Bối Cảnh Vấn Đề:** Sharding là một giải pháp hàng đầu để mở rộng blockchain, nhưng giao tiếp giữa các shard (phân mảnh) là một nút thắt cổ chai lớn. Việc xác minh các giao

dịch từ một shard khác đòi hỏi các shard phải tải về và xử lý một lượng lớn dữ liệu (ví dụ: header của các khối), làm giảm hiệu quả mở rộng và tăng độ trễ. Việc đảm bảo tính nguyên tử (atomicity) cho các giao dịch liên-shard là một thách thức lớn về cả hiệu suất và bảo mật.

- **Đóng Góp Chính của Lý Thuyết:** Bằng chứng không Tri thức, đặc biệt là các biến thể đệ quy hoặc có thể tổng hợp, cho phép nén toàn bộ lịch sử hoặc một phần trạng thái của một shard thành một "bằng chứng" mật mã có kích thước rất nhỏ. Một shard khác có thể xác minh bằng chứng này một cách nhanh chóng mà không cần xử lý lại toàn bộ các giao dịch. Điều này giúp giảm đáng kể băng thông và chi phí tính toán cho việc giao tiếp liên-shard, cho phép hệ thống mở rộng hiệu quả hơn.
- **Nhiệm Vụ của Sinh Viên:**
 1. **Nghiên cứu Lý thuyết:** Tìm hiểu sâu về các thách thức của giao tiếp liên-shard (độ trễ, chi phí xác minh, tính nguyên tử). Nghiên cứu cách zk-SNARKs có thể được sử dụng để tạo ra các bằng chứng ngắn gọn về các phép tính toán.
 2. **Thiết kế Giao thức Đơn giản hóa:** Thiết kế một giao thức mô phỏng giữa hai shard. Shard A muốn chứng minh cho Shard B rằng một tập hợp các giao dịch đã được thực hiện và dẫn đến một cập nhật trạng thái cụ thể (ví dụ: một Merkle root mới).
 3. **Triển khai Mạch ZKP:** Sử dụng một framework như Circom, viết một mạch ZKP để chứng minh việc áp dụng một loạt các giao dịch lên một Merkle root cũ sẽ tạo ra một Merkle root mới.
 4. **Triển khai Hợp đồng Thông minh:** Viết hai hợp đồng thông minh mô phỏng hai shard. Hợp đồng Shard A sẽ quản lý trạng thái của nó. Hợp đồng Shard B (light client) sẽ chứa một hàm `verifyStateUpdateFromShardA` nhận vào một bằng chứng ZKP và Merkle root mới, sau đó xác minh nó.
 5. **Viết Báo cáo:** Tài liệu hóa thiết kế giao thức, chi tiết mạch ZKP, và phân tích các lợi ích về hiệu suất (kích thước bằng chứng so với kích thước dữ liệu gốc, chi phí gas để xác minh) so với các phương pháp giao tiếp liên-shard truyền thống.
- **Kiến thức Nền tảng Yêu cầu:** Hiểu biết sâu về kiến trúc blockchain và các giải pháp mở rộng. Kiến thức vững chắc về ZKP. Kỹ năng lập trình Solidity và Circom/JavaScript.

R5: Kết hợp Mạng Nơ-ron Đồ thị và Phân tích Tĩnh để Tăng cường Phát hiện Lỗ hổng

- **Bài Báo Khoa Học Chính:** [\(PDF\) GraphSA: Smart Contract Vulnerability Detection Combining Graph Neural Networks and Static Analysis](#)
- **Bối Cảnh Vấn Đề:** Các phương pháp phát hiện lỗ hổng chỉ dựa trên GNN có thể bỏ lỡ các ngữ cảnh quan trọng mà các công cụ phân tích tĩnh truyền thống có thể nắm bắt được. Ngược lại, các công cụ phân tích tĩnh thường tạo ra nhiều cảnh báo sai và bị giới hạn bởi các quy tắc được định nghĩa trước. Việc kết hợp sức mạnh của cả hai phương pháp có thể tạo ra một công cụ phát hiện mạnh mẽ và chính xác hơn.
- **Đóng Góp Chính của Bài Báo:** Bài báo đề xuất GraphSA, một phương pháp kết hợp cả hai. Đầu tiên, nó sử dụng các công cụ phân tích tĩnh (như Oyente, Mythril) để tạo ra Đồ

thị Luồng Điều khiển (CFG) từ mã opcode của hợp đồng. Sau đó, CFG này được chuyển đổi thành một "cây hợp đồng" (contract tree) bằng cách loại bỏ và hợp nhất các nút không quan trọng để làm nổi bật các hoạt động và luồng thực thi chính. Cuối cùng, một mô hình GNN chuyên biệt (ST-GNN) được áp dụng trên cây này để học các đặc trưng và phát hiện lỗi hổng.

- **Nhiệm Vụ của Sinh Viên:**

1. **Nghiên cứu Lý thuyết:** Đọc và hiểu sâu phương pháp GraphSA, đặc biệt là quá trình chuyển đổi từ CFG sang "cây hợp đồng" và kiến trúc của mô hình ST-GNN.
 2. **Xây dựng Pipeline Dữ liệu:** Viết một pipeline tự động:
 - Sử dụng một công cụ phân tích tĩnh có sẵn (ví dụ: Slither) để xuất CFG cho một tập hợp các hợp đồng thông minh.
 - Triển khai logic để xử lý và đơn giản hóa các CFG này thành "cây hợp đồng" theo các quy tắc được mô tả trong bài báo.
 3. **Triển khai và Huấn luyện Mô hình:** Cài đặt lại mô hình ST-GNN bằng PyTorch hoặc TensorFlow. Huấn luyện mô hình trên một bộ dữ liệu các hợp đồng thông minh đã được gán nhãn lỗi hổng.
 4. **Đánh giá So sánh:** Đánh giá hiệu suất của mô hình GraphSA. Để làm nổi bật đóng góp, hãy so sánh kết quả với một mô hình GNN cơ bản (như trong đề tài R1) được huấn luyện trên cùng một bộ dữ liệu.
 5. **Viết Báo cáo:** Trình bày chi tiết về pipeline xử lý dữ liệu, kiến trúc mô hình, và kết quả so sánh. Phân tích tại sao việc kết hợp phân tích tĩnh lại cải thiện hiệu suất phát hiện.
- **Kiến thức Nền tảng Yêu cầu:** Kiến thức về Học máy (đặc biệt là GNN), phân tích chương trình (CFG), bảo mật hợp đồng thông minh, Python, PyTorch/TensorFlow.

R6: Phân tích So sánh các Cơ chế Đấu giá MEV (Maximal Extractable Value)

- **Bài Báo Khoa Học Chính:** Dựa trên các nguyên lý từ [Flashbots Auction](#) và [MEV-Auction](#).
- **Bối Cảnh Vấn Đề:** MEV (Giá trị Có thể Trích xuất Tối đa) là lợi nhuận mà các nhà sản xuất khối (miner/validator) có thể kiếm được bằng cách sắp xếp, chen hoặc kiểm duyệt các giao dịch. Cuộc cạnh tranh để khai thác MEV dẫn đến các tiêu cực như chiến tranh gas, tắc nghẽn mạng và các cuộc tấn công sandwich vào người dùng. Các cơ chế đấu giá được đề xuất để tạo ra một thị trường có trật tự và công bằng hơn cho MEV.
- **Đóng Góp Chính của Lý Thuyết:** Flashbots đã tiên phong trong việc tạo ra một kênh giao tiếp riêng tư giữa "searcher" (người tìm kiếm MEV) và nhà sản xuất khối, thông qua một cuộc đấu giá kín giá đầu tiên (first-price sealed-bid auction) cho các "gói" giao dịch (bundles). Điều này giúp giảm tắc nghẽn mạng. Các đề xuất khác, như MEV-Auction, đi xa hơn bằng cách tách biệt vai trò của người đề xuất khối (chọn giao dịch) và người sắp xếp (sắp xếp thứ tự), tạo ra một thị trường rõ ràng cho quyền sắp xếp giao dịch.
- **Nhiệm Vụ của Sinh Viên:**
 1. **Nghiên cứu Lý thuyết:** Tìm hiểu sâu về MEV, các loại hình của nó (arbitrage,

sandwich, liquidation) và các tác động tiêu cực. Nghiên cứu kỹ kiến trúc của Flashbots và thiết kế lý thuyết của MEV-Auction.

2. **Xây dựng Môi trường Mô phỏng:** Phát triển một môi trường mô phỏng đơn giản (ví dụ: bằng Python) để mô hình hóa một mempool, các tác nhân (người dùng, searcher MEV) và quá trình tạo khối.
 3. **Triển khai các Cơ chế Đấu giá:** Cài đặt các phiên bản đơn giản hóa của cả hai cơ chế đấu giá trong môi trường mô phỏng:
 - **Flashbots-style:** Các searcher gửi các gói giao dịch kín, và nhà sản xuất khối chọn gói có lợi nhuận cao nhất.
 - **MEVA-style:** Một cuộc đấu giá riêng để chọn ra một "sequencer", người sau đó có quyền sắp xếp các giao dịch trong khối.
 4. **Thiết kế và Chạy Thí nghiệm:** So sánh hai cơ chế dựa trên các chỉ số quan trọng: tổng doanh thu của nhà sản xuất khối, chi phí giao dịch trung bình của người dùng, mức độ hiệu quả của thị trường, và mức độ công bằng (ví dụ: tỷ lệ các giao dịch của người dùng bị tấn công sandwich).
 5. **Viết Báo cáo:** Trình bày mô hình mô phỏng, thiết kế của các cơ chế đấu giá, và phân tích định lượng về các ưu và nhược điểm của mỗi phương pháp dựa trên kết quả thí nghiệm.
- **Kiến thức Nền tảng Yêu cầu:** Hiểu biết về kinh tế học blockchain (MEV), lý thuyết đấu giá, kỹ năng mô phỏng và phân tích dữ liệu, lập trình Python.

R7: Triển khai zk-ML: Thực thi Mô hình Học máy có thể Kiểm chứng trên Blockchain sử dụng ZKP

- **Bài Báo Khoa Học Chính:** Dựa trên các bài khảo sát và tổng quan về ZKP-ML, ví dụ như [A Survey of Zero-Knowledge Proof Based Verifiable Machine Learning](#)
- **Bối Cảnh Vấn Đề:** Việc tích hợp các mô hình Học máy (ML) vào các ứng dụng blockchain (ví dụ: chấm điểm tín dụng phi tập trung, thị trường dự đoán) gặp phải một rào cản lớn: làm thế nào để tin tưởng vào kết quả của một mô hình được thực thi off-chain mà không cần tiết lộ dữ liệu đầu vào riêng tư của người dùng hoặc các trọng số độc quyền của mô hình?
- **Đóng Góp Chính của Lý Thuyết:** zk-ML (Zero-Knowledge Machine Learning) sử dụng ZKP để tạo ra một bằng chứng mật mã chứng minh rằng một mô hình ML cụ thể đã được thực thi một cách chính xác trên một đầu vào nào đó để tạo ra một đầu ra nhất định. Bằng chứng này có thể được xác minh on-chain mà không tiết lộ bất kỳ thông tin nào về đầu vào hoặc trọng số của mô hình, đảm bảo cả tính toàn vẹn và quyền riêng tư.
- **Nhiệm Vụ của Sinh Viên:**
 1. **Nghiên cứu Lý thuyết:** Tìm hiểu các khái niệm cơ bản về cách chuyển đổi các phép tính toán (bao gồm cả các phép toán trong ML như nhân ma trận, hàm kích hoạt) thành các mạch số học (arithmetic circuits) có thể chứng minh được bằng ZKP.
 2. **Chọn và Đơn giản hóa Mô hình:** Chọn một mô hình ML rất đơn giản, ví dụ như một mạng nơ-ron nhỏ (vài nơ-ron, một lớp ẩn) hoặc một mô hình hồi quy logistic.

3. **Chuyển đổi sang Mạch ZKP:** Đây là phần cốt lõi và thách thức nhất. Sử dụng một framework ZKP (như Circom) để viết một mạch biểu diễn quá trình suy luận (inference) của mô hình ML đã chọn. Điều này có thể đòi hỏi việc lượng tử hóa (quantization) các trọng số và đầu vào.
4. **Xây dựng Hệ thống Proof-of-Concept:**
 - Viết một kịch bản "prover" (bằng JavaScript/Python) để lấy trọng số mô hình và dữ liệu đầu vào, sau đó tạo ra bằng chứng ZKP.
 - Viết một hợp đồng thông minh "verifier" trên Solidity có khả năng xác minh bằng chứng được tạo ra.
5. **Viết Báo cáo:** Trình bày chi tiết quá trình chuyển đổi mô hình ML thành mạch, các thách thức gặp phải (ví dụ: xử lý các phép toán dấu phẩy động, kích thước mạch), và đánh giá hiệu suất (thời gian tạo bằng chứng, chi phí gas để xác minh).
- **Kiến thức Nền tảng Yêu cầu:** Kiến thức vững chắc về cả Học máy và ZKP. Kỹ năng lập trình tốt với các framework ZKP và Solidity. Đây là một đề tài rất khó và mang tính tiên phong.

R8: Mô phỏng và Đánh giá Tấn công Double-Spending trong Giao thức Sharding dựa trên Lưu trữ Đa-Shard

- **Bài Báo Khoa Học Chính:** <https://www.mdpi.com/2076-3417/15/6/3260>
- **Bối Cảnh Vấn Đề:** Tấn công chi tiêu hai lần (double-spending) là một trong những mối đe dọa cơ bản nhất đối với tính toàn vẹn của blockchain. Trong các hệ thống sharding, nguy cơ này trở nên phức tạp hơn, đặc biệt với các giao dịch liên-shard. Kẻ tấn công có thể cố gắng xác nhận một giao dịch chi tiêu trên một shard và một giao dịch xung đột khác trên một shard khác trước khi hệ thống đồng bộ hóa trạng thái.
- **Đóng Góp Chính của Bài Báo:** Bài báo này đề xuất một Giao thức Sharding dựa trên Lưu trữ Đa-Shard (MSSP). Trong đó, một số node có thể lưu trữ dữ liệu từ nhiều shard, tạo thành các "vùng đồng thuận" (consensus zones) có khả năng xử lý trực tiếp các giao dịch liên-shard. Để chống lại tấn công chi tiêu hai lần, giao thức giới thiệu một "cơ chế sắp xếp ưu tiên" (priority sorting mechanism) để quyết định thứ tự của các khối từ các vùng đồng thuận khác nhau.
- **Nhiệm Vụ của Sinh Viên:**
 1. **Nghiên cứu Giao thức:** Đọc và hiểu kỹ lưỡng giao thức MSSP, đặc biệt là cách các vùng đồng thuận được hình thành và cách cơ chế sắp xếp ưu tiên hoạt động để ngăn chặn chi tiêu hai lần.
 2. **Xây dựng Môi trường Mô phỏng:** Phát triển một mô hình mô phỏng sự kiện rời rạc (discrete-event simulation) của một mạng blockchain sharding. Mô hình cần bao gồm các thực thể như node, shard, vùng đồng thuận, và một trình tạo giao dịch.
 3. **Triển khai Logic Giao thức:** Cài đặt các thành phần cốt lõi của MSSP trong môi trường mô phỏng, bao gồm logic lưu trữ đa-shard và cơ chế sắp xếp ưu tiên.
 4. **Thiết kế Tác nhân Tấn công:** Tạo một tác nhân tấn công có khả năng thực hiện một cuộc tấn công chi tiêu hai lần bằng cách gửi các giao dịch xung đột đến các vùng

đồng thuận khác nhau.

5. **Chạy Thí nghiệm và Đánh giá:** Thực hiện các kịch bản mô phỏng để đánh giá khả năng chống chịu của giao thức. Đo lường tỷ lệ thành công của cuộc tấn công dưới các điều kiện mạng khác nhau (ví dụ: độ trễ mạng, tỷ lệ node độc hại).
 6. **Viết Báo cáo:** Trình bày chi tiết thiết kế của môi trường mô phỏng, mô hình tấn công, và các kết quả thực nghiệm. Đưa ra kết luận về hiệu quả của cơ chế phòng thủ được đề xuất trong bài báo.
- **Kiến thức Nền tảng Yêu cầu:** Hiểu biết về bảo mật blockchain, kiến trúc sharding, kỹ năng lập trình mô phỏng (ví dụ: SimPy trong Python).

R9: Phân tích và Tái hiện Mô hình AMM Thanh khoản Tập trung (Concentrated Liquidity)

- **Bài Báo Khoa Học Chính:** Dựa trên kiến trúc của Orca Whirlpools và các nguyên lý nền tảng từ Uniswap V3.
- **Bối Cảnh Vấn Đề:** Các Nhà tạo lập Thị trường Tự động (AMM) truyền thống như Uniswap V2 phân bổ thanh khoản đồng đều trên toàn bộ dải giá từ 0 đến vô cùng. Điều này cực kỳ kém hiệu quả về vốn vì hầu hết các giao dịch chỉ diễn ra trong một phạm vi giá hẹp.
- **Đóng Góp Chính của Lý Thuyết:** AMM Thanh khoản Tập trung (CLMM) cho phép các nhà cung cấp thanh khoản (LP) tập trung vốn của họ vào các phạm vi giá tùy chỉnh. Điều này làm tăng đáng kể hiệu quả sử dụng vốn, dẫn đến độ trượt giá thấp hơn cho các nhà giao dịch và khả năng kiếm được nhiều phí hơn cho các LP (trên mỗi đơn vị vốn). Về cơ bản, nó cho phép các LP tạo ra các đường cong giá của riêng họ, thay vì tuân theo một đường cong $x*y=k$ duy nhất.
- **Nhiệm Vụ của Sinh Viên:**
 1. **Nghiên cứu Lý thuyết:** Nghiên cứu sâu về toán học và kiến trúc đằng sau CLMM. Đọc whitepaper của Uniswap V3 và phân tích mã nguồn mở của các dự án như Orca Whirlpools để hiểu cách các "tick" (vị trí giá), vị thế thanh khoản (dưới dạng NFT) và logic hoán đổi được triển khai.
 2. **Triển khai Mô hình Mô phỏng:** Xây dựng một mô hình mô phỏng của một CLMM bằng Python hoặc JavaScript. Mô hình này không cần phải là một hợp đồng thông minh hoàn chỉnh, nhưng phải tái hiện chính xác logic cốt lõi:
 - Thêm/xóa thanh khoản trong một phạm vi giá cụ thể.
 - Tính toán kết quả của một giao dịch hoán đổi khi nó đi qua các vị thế thanh khoản khác nhau.
 - Theo dõi và phân bổ phí giao dịch cho các vị thế đang hoạt động.
 3. **Phân tích So sánh:** Sử dụng mô hình mô phỏng để thực hiện các thí nghiệm so sánh hiệu quả vốn giữa CLMM và một AMM truyền thống. Ví dụ: với cùng một lượng vốn, CLMM có thể hỗ trợ các giao dịch lớn hơn với độ trượt giá thấp hơn bao nhiêu?
 4. **Viết Báo cáo:** Trình bày chi tiết về lý thuyết của CLMM, kiến trúc của mô hình mô phỏng, và các kết quả phân tích so sánh. Thảo luận về các khái niệm như tổn thất vô thường (impermanent loss) trong bối cảnh thanh khoản tập trung.

- **Kiến thức Nền tảng Yêu cầu:** Hiểu biết sâu về DeFi và cơ chế AMM. Kỹ năng lập trình và mô hình hóa tài chính tốt.

R10: Mã hóa Dựa trên Danh tính Phân cấp Ẩn danh cho việc Thu hồi VC

- **Bài Báo Khoa Học Chính:** [Towards Privacy-Preserving Revocation of Verifiable Credentials with Time-Flexibility](#)
- **Bối Cảnh Vấn Đề:** Đây là một nghiên cứu sâu hơn về cơ chế mật mã được sử dụng trong đề tài R3. Vấn đề cốt lõi vẫn là làm thế nào để thu hồi một Chứng chỉ Xác thực (VC) mà không vi phạm quyền riêng tư của người nắm giữ (Holder).
- **Đóng Góp Chính của Bài Báo:** Bài báo đề xuất sử dụng một công cụ mật mã tiên tiến gọi là Mã hóa Dựa trên Danh tính Phân cấp Ẩn danh (Anonymous Hierarchical Identity-Based Encryption - AHIBE). AHIBE cho phép tạo ra một hệ thống phân cấp các khóa (ví dụ: khóa của trường đại học có thể tạo ra khóa cho các khoa, và khóa của khoa có thể tạo ra khóa cho sinh viên). Thuộc tính "ẩn danh" đảm bảo rằng một bản mã không tiết lộ danh tính của người nhận dự định. Trong bối cảnh thu hồi VC, cơ chế này được sử dụng để cấp cho bên xác minh (Verifier) quyền truy cập có giới hạn thời gian vào thông tin thu hồi, mà không cho phép bên phát hành (Issuer) theo dõi việc sử dụng VC.
- **Nhiệm Vụ của Sinh Viên:**
 1. **Nghiên cứu Mật mã Chuyên sâu:** Đây là một đề tài mang nặng tính lý thuyết. Nhiệm vụ chính là **đọc và hiểu sâu về cấu trúc mật mã của AHIBE**, dựa trên bài báo và các tài liệu tham khảo của nó.
 2. **Đặc tả Giao thức Chính thức:** Thay vì triển khai mã nguồn, sinh viên cần **tạo ra một bản đặc tả chi tiết và chính thức của giao thức thu hồi**. Xác định rõ các tác nhân (PKG, Issuer, Holder, Verifier) và **các thuật toán mật mã trong mỗi bước: Setup, KeyGen, Encrypt, Decrypt, DelegateAccess.**
 3. **Phân tích An ninh và Quyền riêng tư:** Viết một bản phân tích an ninh bán chính thức (semi-formal). Dựa trên các giả định mật mã cơ bản, hãy lập luận và chứng minh cách giao thức đạt được các mục tiêu về quyền riêng tư đã đề ra, chẳng hạn như:
 - Untraceability by Issuer: Bên phát hành không thể biết khi nào và ở đâu VC được trình diện.
 - Untraceability by Verifier: Bên xác minh không thể theo dõi trạng thái của VC ngoài khung thời gian được cho phép.
 4. **Viết Báo cáo Khoa học:** Soạn một báo cáo có cấu trúc như một bài báo khoa học, giải thích chi tiết về nguyên thủy mật mã AHIBE, đặc tả giao thức, và trình bày bản phân tích an ninh. So sánh phương pháp này với các cơ chế thu hồi VC khác.
- **Kiến thức Nền tảng Yêu cầu:** Kiến thức mật mã học nâng cao, khả năng đọc hiểu và phân tích các chứng minh toán học. Kỹ năng viết học thuật tốt.

Nhánh II: Dev Track

Tổng quan các Đề tài Nhánh Phát triển

Mã Đề Tài	Tên Đề Tài	Lĩnh Vực/Công nghệ Cốt Lõi	Framework/Tư viện Gợi ý	Độ Khó Gợi Ý
D1	Ví Danh tính Phi tập trung cho Chứng chỉ Xác thực (VC)	DID, VC, Self-Sovereign Identity	Veramo SDK, React, Ethers.js	Trung bình
D2	Công cụ Trực quan hóa Đồ thị Luồng Điều khiển (CFG) cho Hợp đồng Thông minh	Phân tích Tĩnh, Công cụ Lập trình	@solidity-parser/parser, React, D3.js/vis.js	Trung bình
D3	Sàn Giao dịch Phi tập trung (DEX) dựa trên Sổ lệnh On-chain	DeFi, Sổ lệnh (Order Book)	Solidity, Foundry/Hardhat, Ethers.js	Nâng cao
D4	Hệ thống Đấu giá MEV Công bằng sử dụng Cơ chế Commit-Reveal	DeFi, MEV, Thiết kế Giao thức	Solidity, JavaScript (off-chain)	Nâng cao
D5	AMM cho các Tài sản Kém Thanh khoản sử dụng Thanh khoản Tập trung	DeFi, AMM	Solidity, Vyper (tham khảo)	Nâng cao

D6	Công cụ Mô phỏng và Phân tích MEV trên Dữ liệu Giao dịch Lịch sử	DeFi, MEV, Phân tích Dữ liệu	Python, Web3.py, Pandas	Trung bình
D7	DApp Phát hành và Xác minh Chứng chỉ Tốt nghiệp trên Blockchain	DID, VC, DApp	Veramo SDK, React, Solidity	Trung bình
D8	Plugin Phân tích Mã Tĩnh cho Solidity trên VS Code	Công cụ Lập trình, Phân tích Tĩnh	TypeScript, VS Code Extension API, @solidity-parser/parser	Nâng cao
D9	Cơ chế Khớp lệnh Off-chain và Thanh toán On-chain cho DEX Sổ lệnh	Kiến trúc Hybrid, Scalability	Go/Node.js (matching engine), Solidity, EIP-712	Rất khó
D10	Tiện ích Mở rộng Trình duyệt phát hiện Cơ hội Tấn công Sandwich	DeFi, MEV, Bảo mật Giao dịch	JavaScript, Ethers.js, Browser Extension API	Nâng cao

Chi tiết các Đề tài Phát triển

D1: Ví Danh tính Phi tập trung cho Chứng chỉ Xác thực (VC)

- Mục tiêu và Tầm nhìn:** Xây dựng một ví điện tử đơn giản dưới dạng tiện ích mở rộng cho trình duyệt (ví dụ: Chrome Extension). Ví này cho phép người dùng tạo và quản lý các Định danh Phi tập trung (Decentralized Identifiers - DIDs) và lưu trữ, trình diễn các Chứng chỉ Xác thực (Verifiable Credentials - VCs) theo chuẩn W3C. Dự án này giúp hiện thực

hóa các khái niệm trừu tượng của Danh tính Tự chủ (SSI), cho thấy một tương lai nơi người dùng, chứ không phải các nền tảng lớn như Google hay Facebook, thực sự kiểm soát danh tính số của mình.

- **Các Tính năng Cốt lõi:**

1. **Thiết lập Agent:** Tạo một "agent" Veramo phía client, là trung tâm điều phối mọi hoạt động liên quan đến danh tính.
2. **Quản lý DID:** Cho phép người dùng tạo mới, lưu trữ và quản lý các DID. Hỗ trợ ít nhất một phương thức DID như did:key (đơn giản, dựa trên một cặp khóa) hoặc did:ethr (liên kết với một địa chỉ Ethereum).
3. **Lưu trữ Chứng chỉ:** Cung cấp chức năng để nhận và lưu trữ an toàn một VC. Nhóm cần tạo một dịch vụ "issuer" giả lập đơn giản để cấp phát VC cho ví. Sử dụng các plugin lưu trữ dữ liệu của Veramo (@veramo/data-store hoặc @veramo/data-store-json).
4. **Trình diện Chứng chỉ:** Xây dựng giao diện cho phép người dùng chọn một VC đã lưu và tạo ra một "Trình diện Xác thực" (Verifiable Presentation - VP) để gửi cho một trang web "verifier" giả lập. VP có thể chứa toàn bộ hoặc chỉ một phần thông tin từ VC gốc (selective disclosure).

- **Công nghệ Gợi ý:** React, TypeScript, Veramo SDK (bao gồm các gói @veramo/core, @veramo/did-manager, @veramo/credential-w3c, @veramo/key-manager, @veramo/kms-local).

- **Sản phẩm Bàn giao:** Một kho mã nguồn trên GitHub chứa code của tiện ích mở rộng. Một file README chi tiết hướng dẫn cài đặt và sử dụng. Một video ngắn demo toàn bộ vòng đời: tạo DID, nhận một VC từ issuer, và trình diện VC đó cho verifier.

D2: Công cụ Trực quan hóa Đồ thị Luồng Điều khiển (CFG) cho Hợp đồng Thông minh

- **Mục tiêu và Tầm nhìn:** Tạo một công cụ dựa trên web cho phép người dùng dán mã nguồn Solidity vào và nhận lại một biểu đồ trực quan, tương tác của Đồ thị Luồng Điều khiển (Control Flow Graph - CFG) cho các hàm trong hợp đồng. Đây là một công cụ cơ bản nhưng cực kỳ hữu ích cho các nhà phân tích bảo mật và lập trình viên để hiểu rõ logic thực thi của một hợp đồng. Việc trừu tượng hóa mã nguồn thành các cấu trúc như đồ thị là bước đầu tiên và quan trọng nhất cho các phân tích phức tạp hơn, bao gồm cả việc áp dụng các mô hình học máy như GNN.¹ Công cụ này chính là cầu nối giữa mã nguồn và các phương pháp phân tích nâng cao đó.

- **Các Tính năng Cốt lõi:**

1. **Giao diện Nhập liệu:** Một giao diện web đơn giản với một trình soạn thảo văn bản (code editor) để người dùng có thể dán mã nguồn Solidity.
2. **Phân tích Cú pháp (Parsing):** Sử dụng một thư viện parser Solidity cho JavaScript như @solidity-parser/parser để chuyển đổi mã nguồn thành một Cây Cú pháp Trừu tượng (Abstract Syntax Tree - AST).
3. **Xây dựng CFG:** Viết logic để duyệt cây AST, xác định các hàm, các khối cơ bản

(basic blocks - một chuỗi các lệnh không có lệnh nhảy vào hoặc nhảy ra ở giữa), và các điểm chuyển giao luồng điều khiển (như if/else, for, while, require, revert, và các lệnh gọi hàm).

4. **Trực quan hóa:** Sử dụng một thư viện trực quan hóa đồ thị (ví dụ: D3.js, vis.js, React Flow) để vẽ CFG. Các nút (nodes) sẽ là các khối cơ bản, và các cạnh (edges) sẽ biểu diễn các đường đi có thể có của luồng thực thi.
 5. **Tính Tương tác:** Cho phép người dùng nhấp vào một nút trên đồ thị để làm nổi bật các dòng mã tương ứng trong trình soạn thảo.
- **Công nghệ Gợi ý:** React/Vue/Svelte, @solidity-parser/parser, một thư viện trực quan hóa đồ thị.
 - **Sản phẩm Bàn giao:** Một ứng dụng web có thể truy cập công khai. Một kho mã nguồn trên GitHub. Tài liệu giải thích logic duyệt cây AST và xây dựng CFG.

D3: Sàn Giao dịch Phi tập trung (DEX) dựa trên Sổ lệnh On-chain

- **Mục tiêu và Tầm nhìn:** Triển khai một hợp đồng thông minh hoạt động như một Sổ lệnh Giới hạn Trung tâm (Central Limit Order Book - CLOB) hoàn toàn on-chain trên một mạng thử nghiệm EVM. Trong khi các Nhà tạo lập Thị trường Tự động (AMM) thống trị DeFi, chúng có những nhược điểm như tổn thất vô thường (impermanent loss) và trượt giá cao, đặc biệt với các tài sản kém thanh khoản. Sổ lệnh, mô hình của các sàn giao dịch truyền thống, cung cấp khả năng khám phá giá tốt hơn và kiểm soát giao dịch chính xác hơn. Dự án này buộc sinh viên phải đối mặt với các thách thức kinh điển của khoa học máy tính (như cấu trúc dữ liệu hiệu quả) trong môi trường blockchain, nơi mỗi thao tác đều tốn gas.
- **Các Tính năng Cốt lõi:**
 1. **Đặt lệnh:** Các hàm trong hợp đồng thông minh cho phép người dùng đặt các lệnh giới hạn (limit orders) mua và bán cho một cặp token ERC20. Các lệnh này phải được lưu trữ trong một cấu trúc dữ liệu được sắp xếp (ví dụ: danh sách liên kết đôi có sắp xếp, cây nhị phân tự cân bằng, hoặc skip list) để tối ưu hóa chi phí gas khi tìm kiếm và khớp lệnh.
 2. **Khớp lệnh:** Khi một lệnh mới được đặt, hợp đồng phải tự động tìm kiếm và khớp nó với các lệnh đối ứng đang chờ trong sổ lệnh. Logic khớp lệnh phải xử lý cả trường hợp khớp một phần và khớp toàn bộ.
 3. **Hủy lệnh:** Cho phép người dùng hủy các lệnh chưa được khớp của chính họ.
 4. **Quản lý Tài sản:** Hợp đồng phải có khả năng giữ và chuyển giao tài sản (cả hai token ERC20) một cách an toàn khi một giao dịch được thực hiện thành công.
- **Công nghệ Gợi ý:** Solidity, Foundry hoặc Hardhat để kiểm thử và triển khai, Ethers.js và một giao diện React đơn giản để tương tác với hợp đồng.
- **Sản phẩm Bàn giao:** Các hợp đồng thông minh đã được triển khai trên một mạng thử nghiệm (ví dụ: Sepolia). Một kho mã nguồn trên GitHub với bộ kiểm thử toàn diện (unit tests). Một giao diện web đơn giản để người dùng có thể đặt lệnh, hủy lệnh và xem trạng thái sổ lệnh.

D4: Hệ thống Đấu giá MEV Công bằng sử dụng Cơ chế Commit-Reveal

- **Mục tiêu và Tầm nhìn:** Xây dựng một DApp đấu giá kín (sealed-bid auction) on-chain để giảm thiểu MEV, đặc biệt là front-running, trong các sự kiện cạnh tranh cao như mint NFT hoặc bán token. Thay vì để người tham gia thấy các giá thầu của nhau và tham gia vào cuộc chiến gas, cơ chế commit-reveal che giấu thông tin giá thầu cho đến khi không thể thay đổi được nữa.
- **Các Tính năng Cốt lõi:**
 1. **Giai đoạn Commit:** Người dùng gửi một khoản tiền đặt cọc cùng với một hash của giá thầu thực sự của họ (`keccak256(abi.encodePacked(bidValue, nonce))`) đến hợp đồng thông minh. nonce là một số ngẫu nhiên để ngăn chặn tấn công đoán giá thầu.
 2. **Giai đoạn Reveal:** Sau khi giai đoạn commit kết thúc, người dùng gửi giá thầu bidValue và nonce ban đầu của họ. Hợp đồng sẽ xác minh hash này có khớp với hash đã commit hay không.
 3. **Xử lý Kết quả:** Sau khi giai đoạn reveal kết thúc, một hàm `finalizeAuction` sẽ được gọi để xác định người chiến thắng (ví dụ: người trả giá cao nhất). Hợp đồng sẽ chuyển tài sản đấu giá cho người thắng và cho phép những người thua cuộc rút lại tiền đặt cọc.
 4. **Giao diện Người dùng:** Một giao diện web đơn giản hướng dẫn người dùng qua hai giai đoạn của cuộc đấu giá.
- **Công nghệ Gợi ý:** Solidity, Hardhat/Foundry, Ethers.js, React.
- **Sản phẩm Bàn giao:** Một kho mã nguồn GitHub chứa hợp đồng thông minh và giao diện người dùng. Hợp đồng được triển khai trên một mạng thử nghiệm. Một video demo quá trình một người dùng tham gia và thắng cuộc đấu giá.

D5: AMM cho các Tài sản Kém Thanh khoản sử dụng Thanh khoản Tập trung

- **Mục tiêu và Tầm nhìn:** Xây dựng một sàn giao dịch phi tập trung (DEX) sử dụng mô hình Thanh khoản Tập trung (Concentrated Liquidity) đơn giản hóa. Mục tiêu là tạo ra một thị trường hiệu quả hơn cho các cặp token "long-tail" (kém thanh khoản), nơi mà các AMM truyền thống thường có độ trượt giá rất cao.
- **Các Tính năng Cốt lõi:**
 1. **Quản lý Vị thế (Position Management):** Cho phép các nhà cung cấp thanh khoản (LP) gửi token vào một pool và chỉ định một phạm vi giá (ví dụ: từ \$1 đến \$1.2) mà họ muốn cung cấp thanh khoản. Mỗi vị thế sẽ được đại diện bằng một token NFT.
 2. **Logic Hoán đổi (Swapping Logic):** Triển khai logic hoán đổi có thể xử lý các giao dịch đi qua nhiều vị thế thanh khoản tập trung khác nhau. Khi giá di chuyển ra ngoài phạm vi của một vị thế, thanh khoản đó sẽ trở nên không hoạt động.
 3. **Thu phí (Fee Collection):** Phí giao dịch sẽ được tích lũy cho các vị thế thanh khoản đang hoạt động. Các LP có thể nhận phí của mình bất cứ lúc nào.
 4. **Giao diện Người dùng:** Một giao diện web cho phép các nhà giao dịch hoán đổi token và các LP quản lý (thêm, xóa, xem) các vị thế thanh khoản của họ.

- **Công nghệ Gợi ý:** Solidity (nghiên cứu mã nguồn của Uniswap V3 hoặc Orca Whirlpools là rất quan trọng), Foundry, The Graph để lập chỉ mục dữ liệu on-chain, React, Ethers.js.
- **Sản phẩm Bàn giao:** Các hợp đồng thông minh đã được triển khai. Một DApp hoạt động cho phép hoán đổi và quản lý thanh khoản. Kho mã nguồn GitHub với tài liệu và bộ kiểm thử đầy đủ.

D6: Công cụ Mô phỏng và Phân tích MEV trên Dữ liệu Giao dịch Lịch sử

- **Mục tiêu và Tầm nhìn:** Xây dựng một công cụ dòng lệnh (CLI) bằng Python có khả năng "phát lại" các khối đã được xác nhận trên Ethereum để phát hiện và định lượng các cơ hội MEV đã bị bỏ lỡ hoặc đã được thực hiện. Công cụ này sẽ hữu ích cho các nhà nghiên cứu và các "searcher" MEV để phân tích các chiến lược.
- **Các Tính năng Cốt lõi:**
 1. **Trích xuất Dữ liệu:** Kết nối với một node Ethereum (archive node) thông qua RPC để lấy dữ liệu chi tiết của một khối hoặc một dải khối, bao gồm tất cả các giao dịch theo đúng thứ tự.
 2. **Mô phỏng Trạng thái:** Duy trì một trạng thái mô phỏng đơn giản của các AMM chính (ví dụ: Uniswap V2, V3). Khi "phát lại" các giao dịch trong một khối, công cụ sẽ cập nhật trạng thái của các AMM này.
 3. **Logic Phát hiện MEV:**
 - **Chênh lệch giá (Arbitrage):** Sau mỗi giao dịch, kiểm tra xem có sự chênh lệch giá đáng kể giữa các AMM mô phỏng hay không và tính toán lợi nhuận tiềm năng.
 - **Tấn công Sandwich:** Tìm kiếm các giao dịch hoán đổi lớn trong khối và mô phỏng việc chen một giao dịch mua ngay trước và một giao dịch bán ngay sau nó để xem liệu có thể tạo ra lợi nhuận hay không.
 4. **Báo cáo Kết quả:** Xuất ra một báo cáo tóm tắt các cơ hội MEV được tìm thấy, bao gồm loại MEV, lợi nhuận tiềm năng, và các giao dịch liên quan.
- **Công nghệ Gợi ý:** Python, Web3.py, Pandas, có thể tham khảo các công cụ mã nguồn mở như mev-inspect-py.
- **Sản phẩm Bàn giao:** Một công cụ CLI trong kho mã nguồn GitHub. File README hướng dẫn cách cài đặt, kết nối với node và chạy phân tích trên một dải khối cụ thể.

D7: DApp Phát hành và Xác minh Chứng chỉ Tốt nghiệp trên Blockchain

- **Mục tiêu và Tầm nhìn:** Xây dựng một ứng dụng web đầy đủ chức năng cho phép một tổ chức giáo dục (ví dụ: UIT) cấp bằng tốt nghiệp dưới dạng Chứng chỉ Xác thực (VC) cho sinh viên. Sinh viên có thể lưu trữ và trình diện bằng cấp này cho nhà tuyển dụng một cách an toàn và có thể xác minh bằng mật mã.
- **Các Tính năng Cốt lõi:**
 1. **Cổng thông tin của Tổ chức phát hành (Issuer):** Một trang web được bảo vệ bằng mật khẩu, nơi quản trị viên của trường có thể đăng nhập, điền thông tin sinh viên

- (tên, ngành, ngày tốt nghiệp) và cấp một VC đã ký.
2. **Tích hợp Ví của Người nắm giữ (Holder):** Sinh viên có thể kết nối ví danh tính của mình (có thể là một ví đơn giản được xây dựng trong trình duyệt cho dự án này) để nhận và lưu trữ VC.
 3. **Cổng thông tin của Bên xác minh (Verifier):** Một trang công khai nơi nhà tuyển dụng có thể yêu cầu xác minh bằng cấp. Điều này có thể được thực hiện bằng cách quét mã QR, kích hoạt ví của sinh viên để tạo ra một Trình diện Xác thực (VP).
 4. **Logic Xác minh:** Cổng thông tin nhận VP và tự động kiểm tra chữ ký của cả sinh viên và trường đại học để xác nhận tính hợp lệ của bằng cấp.
- **Công nghệ Gợi ý:** Veramo SDK , React, Node.js/Express cho backend của issuer, Solidity (để neo các DID lên blockchain), Ethers.js.
 - **Sản phẩm Bàn giao:** Một DApp hoạt động được triển khai trực tuyến. Kho mã nguồn GitHub. Một video demo chi tiết luồng công việc của cả ba vai trò: Issuer, Holder, và Verifier.

D8: Plugin Phân tích Mã Tĩnh cho Solidity trên VS Code

- **Mục tiêu và Tầm nhìn:** Phát triển một tiện ích mở rộng cho Visual Studio Code để cung cấp cho các lập trình viên Solidity phản hồi phân tích tĩnh đơn giản và tức thời ngay trong khi họ viết mã. Điều này giúp phát hiện các lỗi phổ biến và các phương pháp lập trình không an toàn từ sớm.
- **Các Tính năng Cốt lõi:**
 1. **Phân tích Cây Cú pháp Trừu tượng (AST):** Khi tệp được lưu hoặc nội dung thay đổi, tiện ích mở rộng sẽ phân tích mã Solidity thành một AST bằng cách sử dụng một thư viện như @solidity-parser/parser.
 2. **Kiểm tra dựa trên Quy tắc:** Triển khai một số quy tắc phân tích tĩnh đơn giản bằng cách duyệt qua AST. Ví dụ:
 - Phát hiện việc sử dụng tx.origin để xác thực.
 - Cảnh báo về việc sử dụng phiên bản pragma không cố định (ví dụ: ^0.8.0).
 - Đánh dấu các hàm thiếu chỉ định phạm vi truy cập (visibility).
 - Tìm kiếm các mẫu có thể dẫn đến tấn công tái nhập (reentrancy).
 3. **Hiển thị Chẩn đoán:** Sử dụng VS Code API để làm nổi bật các dòng mã có vấn đề và hiển thị các thông báo cảnh báo/lỗi trong bảng "Problems" của trình soạn thảo.
- **Công nghệ Gợi ý:** TypeScript, VS Code Extension API, @solidity-parser/parser.
- **Sản phẩm Bàn giao:** Một tệp tiện ích mở rộng VS Code (.vsix) có thể cài đặt. Một kho mã nguồn GitHub. Một file README với hướng dẫn cài đặt và sử dụng, kèm theo ảnh GIF minh họa.

D9: Cơ chế Khớp lệnh Off-chain và Thanh toán On-chain cho DEX Sổ lệnh

- **Mục tiêu và Tầm nhìn:** Xây dựng một sàn giao dịch phi tập trung "lai" (hybrid) kết hợp

tốc độ của một công cụ khớp lệnh tập trung với tính bảo mật của việc thanh toán on-chain. Điều này mang lại trải nghiệm người dùng giống như sàn giao dịch tập trung (giao dịch nhanh, không tốn gas cho mỗi lần đặt/hủy lệnh) nhưng vẫn đảm bảo người dùng tự quản lý tài sản của mình.

- **Các Tính năng Cốt lõi:**

1. **Hợp đồng Ký quỹ (Escrow) On-chain:** Một hợp đồng thông minh nơi người dùng gửi tài sản họ muốn giao dịch. Hợp đồng này hoạt động như một kho lưu trữ an toàn.
2. **Công cụ Khớp lệnh (Matching Engine) Off-chain:** Một máy chủ trung tâm (viết bằng Node.js, Go, hoặc Rust) duy trì sổ lệnh. Người dùng gửi các lệnh đã được ký bằng mật mã (sử dụng EIP-712 để thể hiện rõ ý định) đến máy chủ này thông qua API. Công cụ sẽ khớp các lệnh mua và bán.
3. **Thanh toán (Settlement) On-chain:** Khi một cặp lệnh được khớp, công cụ off-chain sẽ gửi cả hai lệnh đã ký đến hợp đồng ký quỹ. Hợp đồng sẽ xác minh cả hai chữ ký và thực hiện việc hoán đổi tài sản một cách nguyên tử giữa số dư của hai người dùng trong hợp đồng.
4. **Rút tiền (Withdrawals):** Người dùng có thể rút tài sản của họ khỏi hợp đồng ký quỹ bất cứ lúc nào.

- **Công nghệ Gợi ý:** Solidity, Node.js/Go, Redis hoặc cơ sở dữ liệu trong bộ nhớ cho sổ lệnh, Ethers.js, EIP-712.
- **Sản phẩm Bàn giao:** Hợp đồng thanh toán đã được triển khai trên mạng thử nghiệm. Kho mã nguồn GitHub chứa cả mã hợp đồng thông minh và mã công cụ khớp lệnh off-chain. Tài liệu API cho công cụ khớp lệnh.

D10: Tiềm ích Mở rộng Trình duyệt phát hiện Cơ hội Tấn công Sandwich

- **Mục tiêu và Tầm nhìn:** Tạo một tiềm ích mở rộng trình duyệt (tương tự như một tính năng trong MetaMask) để giám sát các giao dịch DeFi mà người dùng sắp gửi và cảnh báo họ nếu giao dịch đó có nguy cơ cao bị tấn công sandwich.
- **Các Tính năng Cốt lõi:**
 1. **Giám sát Mempool:** Tiềm ích mở rộng cần kết nối với một dịch vụ cung cấp luồng dữ liệu thời gian thực về các giao dịch đang chờ trong mempool (ví dụ: Blocknative, Alchemy).
 2. **Mô phỏng Giao dịch:** Khi người dùng chuẩn bị gửi một giao dịch (ví dụ: một lệnh hoán đổi trên Uniswap), tiềm ích mở rộng sẽ chặn lại. Nó sẽ mô phỏng giao dịch đó với trạng thái khối hiện tại để xác định số lượng token đầu ra dự kiến.
 3. **Logic Phát hiện Sandwich:** Tiềm ích mở rộng sẽ quét mempool để tìm các giao dịch có thể "kẹp" giao dịch của người dùng. Nó sẽ mô phỏng một kịch bản "sandwich": (1) giao dịch front-run, (2) giao dịch của người dùng, (3) giao dịch back-run.
 4. **Cảnh báo Người dùng:** Nếu kịch bản sandwich mô phỏng cho thấy người dùng sẽ nhận được số lượng token đầu ra ít hơn đáng kể so với dự kiến (tức là độ trượt giá do tấn công cao), tiềm ích mở rộng sẽ hiển thị một cảnh báo nổi bật trước khi người dùng xác nhận giao dịch.

- **Công nghệ Gợi ý:** JavaScript, API Tiện ích mở rộng Trình duyệt (Browser Extension APIs), Ethers.js, API của một nhà cung cấp dữ liệu mempool.
- **Sản phẩm Bàn giao:** Một tiện ích mở rộng trình duyệt hoạt động trong một kho mã nguồn GitHub. File README giải thích chức năng và cách cài đặt ở chế độ nhà phát triển. Một video demo tiện ích mở rộng đang cảnh báo người dùng về một cuộc tấn công sandwich tiềm tàng.

Hướng dẫn và Quy trình Thực hiện

Để đảm bảo quá trình thực hiện đồ án diễn ra suôn sẻ và công bằng, các em cần tuân thủ các quy định sau:

- **Thành lập Nhóm:** Mỗi nhóm bao gồm **đúng 2 sinh viên**. Các em tự do lựa chọn thành viên nhóm.
- **Đăng ký Đề tài:** Danh sách đề tài sẽ được mở để đăng ký vào một thời điểm cụ thể sẽ được thông báo. Việc đăng ký sẽ tuân theo nguyên tắc "ai đăng ký trước được chọn trước" (first-come, first-served) thông qua một biểu mẫu trực tuyến. Mỗi đề tài chỉ có một nhóm thực hiện.
- **Các Cột mốc (Milestones):** Quá trình thực hiện đồ án sẽ được chia thành các cột mốc để theo dõi tiến độ:
 1. **Cột mốc 1:** Nộp kế hoạch dự án chi tiết và tổng quan tài liệu (literature review cho nhánh Nghiên cứu) hoặc thiết kế kiến trúc (cho nhánh Phát triển).
 2. **Cột mốc 2:** Hoàn thành phần lõi của việc triển khai (core implementation). Demo tiến độ với giảng viên.
 3. **Báo cáo và Nộp cuối kỳ:** Nộp báo cáo hoàn chỉnh, kho mã nguồn và video demo sản phẩm.

Chúc các em có một kỳ đồ án thành công và gặt hái được nhiều kiến thức quý báu!